

BCSPWEA Website — Build Roadmap

PART A — SESSION/TOOLING SETUP (DO FIRST, APPROVED VIA ASKUSERQUESTION)

Three housekeeping items, confirmed with the user, to reduce prompt friction and token waste for this project going forward:

1. Permissions allowlist

Add to `d:\BCSPWEA_website\.claude\settings.local.json`: safe, non-destructive dev-loop commands (`git status / diff / log / branch`, `npm install / run *`, `npx *` for next/prisma/playwright, `ls /directory listing equivalents`). Explicitly exclude anything destructive (push, force flags, reset --hard, rm -rf) — those stay prompted.

2. Document the resolved stack decision in-repo

Add a short, clearly-dated note to `project_state.md` (and/or a pointer in `AGENTS.md`) stating: this project uses Next.js + TypeScript + Prisma + MySQL (via Laragon), NOT the Laravel/Blade stack described in `BCSPWEA_Dev_Guide_for_AI.md`; that doc's DB schema/business rules/security/design-system sections remain authoritative, only its PHP/Laravel framework specifics are superseded. Goal: any future session (with or without my memory) gets this instantly instead of re-deriving it from two conflicting 300+/600+ line docs.

3. Clean up repo clutter

Remove redundant zip files in `Stich UI/` (`files.zip`, `stitch_bilingual_member_portal.zip`, `[old]stitch_bilingual_member_portal.zip` — all duplicate the already-extracted folder) and move/delete the 4 stray `Screenshot*.png` files at repo root. List exact files and confirm before deleting.

CONTEXT

BCSPWEA (BCS Public Works Engineers' Association) needs a full bilingual (Bangla/English) association-management website: public info site + member portal + payments + admin/superadmin panel, per `BCSPWEA_Website_Specification.md` and `BCSPWEA_Dev_Guide_for_AI.md`.

Two planning docs disagreed on stack (Laravel/MySQL vs Next.js/Postgres). User confirmed (2026-07-10): **keep the existing Next.js scaffold, pair it with Prisma + MySQL**, using Laragon (`root`, no password) as the local DB server. The 90+ Google Stitch HTML exports in `Stich UI/` are **visual reference only** — not code to port verbatim, and their "90-95% complete" roadmap claims describe mockup coverage, not real app progress (real app = still the default `create-next-app` page). This is recorded in memory ([[stack-decision]], [[stitch-ui-reference]]).

The Dev Guide's **design system** (Section 3 colors/type/components), **DB schema** (Section 5 — maps to Prisma models), **business rules** (Section 14), **security requirements** (Section 10), and **route/page list** (Section 7) remain authoritative; only its PHP/Laravel-specific language is superseded.

Goal of this task: produce the sequenced roadmap only (no code yet) — implementation happens phase by phase in later sessions, each phase re-confirmed before starting.

ROADMAP

Phase 0 — Foundation & Environment

- Rename `package.json` name from placeholder `app-temp` to `bcspwea-website`.
- Install and configure: Prisma, `@prisma/client`, `mysql2` driver path, NextAuth (or Auth.js) for session-based role auth, `zod` for validation, `next-intl` (or similar) for bn/en routing, shadcn/ui component primitives, `resend` /nodemailer for email, a PDF lib (receipts/certificates), an Excel lib (bulk import/export).
- Create `.env.local` with `DATABASE_URL` pointing at Laragon MySQL (`mysql://root:@localhost:3306/bcspwea`) — **need the DB name confirmed / created in Laragon first**.
- Set up folder structure: `src/app/(public)`, `src/app/(member)`, `src/app/(admin)`, `src/lib`, `src/server` (services), `prisma/schema.prisma`, `messages/bn.json` + `messages/en.json`.
- Establish coding conventions doc (adapted from Dev Guide Section 11 for TS/Next.js): thin route handlers, business logic in `src/server/services/*`, Zod schemas per form, consistent API response shape `{ success, data, message } / { success, errors, message }`.

Phase 1 — Database Schema & Seed Data

- Translate Dev Guide Section 5 tables 1:1 into `prisma/schema.prisma` models: `Member`, `Payment`, `Notice`, `Event`, `EventRegistration`, `EventMedia`, `CommitteeMember`, `PastLeader`, `Document`, `ActivityLog`, `Setting`.
- Run `prisma migrate dev` against Laragon MySQL to create the schema.
- Write a seed script: sample roles/members, notices, events, committee — enough to build UI against real data immediately.

Phase 2 — Authentication & Role Middleware

- Login by `member_code` or email + password (bcrypt via NextAuth Credentials provider).
- Role-based middleware/guards for Guest/Member/Editor/Admin/Superadmin per the Section 4 permission matrix.
- Rate limiting on login (5/min/IP) and registration submit (3/hour/IP).
- Password reset flow (token-based, email delivery).

Phase 3 — Backend Logic (Services, no UI polish)

Build one module at a time as services + API routes, each covering the business rules in Dev Guide Section 6/14:

1. Member registration + approval workflow (`member_code` generation on approval: `PWEA- {YEAR} - {seq}`)
2. Notice module (bn/en fields, visibility, scheduled publish, "new" badge logic)
3. Event module (capacity checks, free-vs-paid branching, registrations)
4. Member directory (filtered search, visibility rules)
5. Document library (auth-gated downloads via controller, never direct URL)
6. Committee & past-leaders modules
7. Activity log writer (wrap all admin mutations)
8. Site settings module

Phase 4 — Payment Module

- `PaymentService` : `initiatePayment()` , `verifyPayment()` , `handleCallback()` , reconciliation job (cron every 60 min for stale pending payments).
- Gateways: bKash, Nagad, SSLCommerz — **need sandbox API credentials from you before this phase**; will stub/mock behind an interface until keys are available so the rest of the app isn't blocked.
- Manual payment path (bank slip upload + admin verification).
- Waived-payment path per business rule (creates a `completed` , `amount=0` record).

Phase 5 — Frontend UI (React/Next.js, using Stitch as visual reference)

Build pages in the package groupings from `bcsppwea_export_guide_package_list.md` (repurposed as a build order, not a completion record):

1. Public site: Homepage, About, Committee, Past Leaders, Notice Board, Gallery, Calendar, Contact
 2. Member access: Login, Forgot Password (OTP), Registration (4-step)
 3. Member portal: Dashboard, Profile, Directory, Documents, Members-only Notices, Event Registration, Payments
 4. Admin/Superadmin: Dashboard, Member Management, Registration Queue, Notice/Event/Committee/Document management, Payment Management, Roles, Activity Logs, Settings
- Apply the Institutional Excellence design tokens (■ navy `#0D2B5E` , ■ gold `#C49A0A` , etc.) via Tailwind config / shadcn theme — not the raw Stitch HTML.
 - Bilingual via `next-intl` , Bangla default, Noto Sans Bengali + Inter fonts.

Phase 6 — Integration Testing

- Playwright (already installed) end-to-end flows: registration → approval → payment → active member; notice publish → visibility rules; event registration capacity limits; role-gated route access.

Phase 7 — Security Hardening & Audit

- CSRF (Next.js server actions/route handlers), file upload MIME validation, private file serving via authenticated route only, payment callback signature verification, HTTPS enforcement, audit of Section 10 checklist.

Phase 8 — Deployment

- **Need to confirm hosting target** (Vercel vs Bangladesh-based VPS vs other) and production DB before this phase.
- Environment/secrets setup, SSL, production Prisma migration strategy.

Phase 9 — Content Entry & Launch

- Real committee/member/notice data entry, final bilingual/accessibility pass, go-live.

IMMEDIATE NEXT STEPS (ONCE THIS PLAN IS APPROVED)

1. Confirm the Laragon MySQL database name to use (or I create one, e.g. `bcsppwea`).
2. Start Phase 0: rename package.json, install dependencies, scaffold folders, write `prisma/schema.prisma` from Section 5 and get it migrating against your Laragon MySQL instance.

3. Re-check in with you before Phase 4 (payment gateways) since it needs real/sandbox credentials, and before Phase 8 (hosting).

VERIFICATION PER PHASE

Phase	Verification
Phase 0/1	<code>npx prisma migrate dev</code> succeeds, <code>npx prisma studio</code> shows seeded tables.
Phase 2	Manual login as seeded users of each role, confirm correct redirects/guards.
Phase 3/4	Unit/integration tests per service (money + auth paths get explicit tests per Dev Guide Section 11).
Phase 5	Visually verify each page against Stitch reference at desktop/tablet/mobile breakpoints in a real browser (per <code>/verify</code> skill).
Phase 6	Playwright suite green.